Algorithms

Week 4 – Divide and Conquer Algorithms

Korea University Sejong Campus, Dept. of Computer Science & Software

Part 1. Divide and Conquer Concept

Divide and Conquer Paradigm

An algorithm that **divides** the input of a given problem and **conquers** (solves) each part.

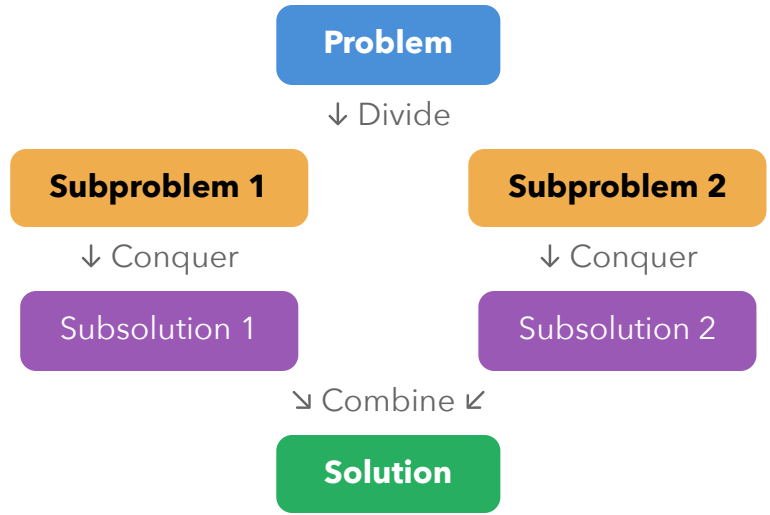
Three steps:

1. **Divide** – Split the problem into smaller subproblems
2. **Conquer** – Solve each subproblem recursively
3. **Combine** – Merge the subsolutions into the solution for the original problem

Key terms:

- **Subproblem**: A problem defined on the divided input
- **Subsolution**: The solution to a subproblem
- Subproblems are divided recursively until they can no longer be split (base case)

DaC Conceptual Diagram



The Divide Phase

Example: Input size n, split into 2 subproblems, each of size $n/2$

After each split, subproblem size halves:

- After 1 split: each size $= n/2$
- After 2 splits: each size $= n/2^2$
- …
- After k splits: each size $= n/2^k$

When does splitting stop?

$$n/2^k = 1 \implies k = \log_2 n$$

Total number of divisions $= \log_2 n$

Conquer and Combine

- Simply dividing input is **not enough** to solve most problems
- We must **conquer** the subproblems: find subsolutions
- **Combine** (merge) subsolutions to build solutions for larger subproblems

The method of conquering depends on the specific problem.

Classification of DaC Algorithms

General recurrence form:

$$T(n) = a \cdot T(n/b) + O(f(n))$$

- a = number of subproblems after division
- n/b = size of each subproblem
- $f(n)$ = cost of dividing and combining

a	b	Algorithm
2	2	Merge sort, Closest pair
3	2	Large integer multiplication
7	2	Strassen's matrix multiplication

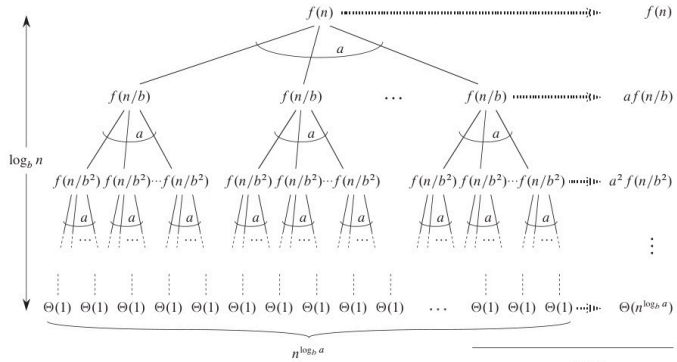
Master Theorem – Recursion Tree Intuition

For $T(n) = a \cdot T(n/b) + O(f(n))$:

Cost at each level of the recursion tree:

Level	Problem size	# of problems	Cost
0 (root)	n	1	$f(n)$
1	n/b	a	$a \cdot f(n/b)$
k	n/b^k	a^k	$a^k \cdot f(n/b^k)$

Depth: $k = \log_b n$ **Leaves:** $n^{\log_b a}$



Master Theorem – Three Cases

For $T(n) = a \cdot T(n/b) + O(f(n))$, compare $f(n)$ with $n^{\log_b a}$:

Case 1: $f(n) = O(n^{\log_b a - \varepsilon})$ – leaf cost dominates

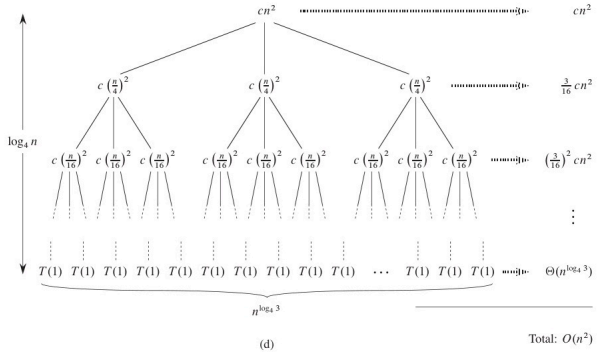
$$T(n) = \Theta(n^{\log_b a})$$

Case 2: $f(n) = \Theta(n^{\log_b a})$ – costs balanced

$$T(n) = \Theta(n^{\log_b a} \log n)$$

Case 3: $f(n) = \Omega(n^{\log_b a + \varepsilon})$ – combine cost dominates

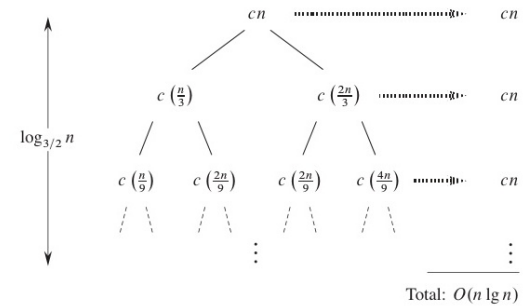
$$T(n) = \Theta(f(n))$$



Note (Case 3): Also requires the *regularity condition*: $a \cdot f(n/b) \leq c \cdot f(n)$ for some $c < 1$.

Other DaC Recurrence Patterns

Recurrence	Description	Example
$T(n) = \frac{1}{n}\sum[T(i) + T(n-i)] + O(?)$	2 parts, unequal sizes	Quick sort
$T(n) = T(n/2) + O(?)$	2 parts, only 1 needed, half size	Binary search
$T(n) = \max\{T(i), T(n-i)\} + O(?)$	2 parts, only 1 needed, unequal	Selection
$T(n) = T(n-1) + O(?)$	Size decreases by 1	Insertion sort, Fibonacci



Merge Sort

DaC Example 1

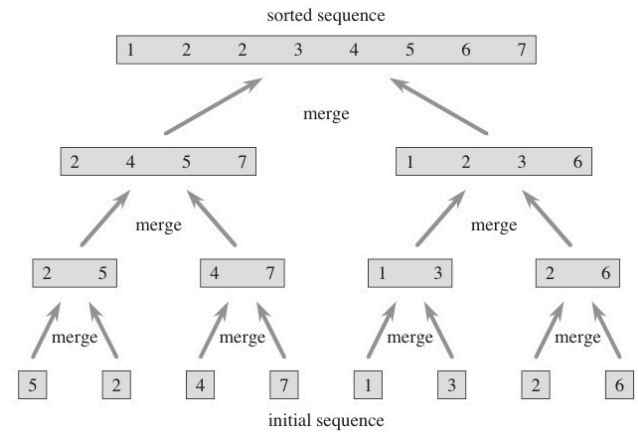
Merge Sort – Overview

Divide: Split array of n elements into two halves of $n/2$

Conquer: Recursively sort each half

Combine: Merge two sorted halves into one sorted array

```
MERGE-SORT(A, p, r)
  if p < r
    q = floor((p + r) / 2)
    MERGE-SORT(A, p, q)       // sort left half
    MERGE-SORT(A, q+1, r)     // sort right half
    MERGE(A, p, q, r)         // merge both halves
```



CLRS Figure 2.4

Merge Sort – Recurrence and Analysis

Recurrence:

$$T(n) = 2T(n/2) + \Theta(n)$$

- Divide: $O(1)$ – just compute the midpoint
- Conquer: $2T(n/2)$ – two recursive calls on halves
- Combine: $\Theta(n)$ – merge two sorted arrays

Applying Master Theorem: $a = 2, b = 2, f(n) = \Theta(n)$

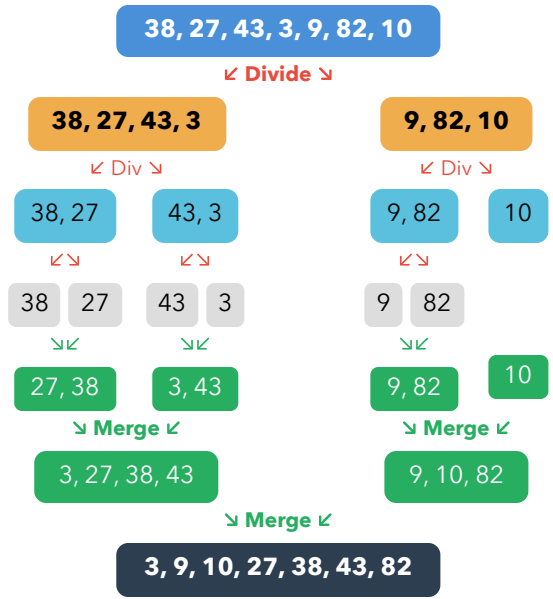
$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

$$f(n) = \Theta(n) = \Theta(n^{\log_b a}) \implies \text{Case 2}$$

$$T(n) = \Theta(n \log n)$$

Space complexity: $O(n)$ (auxiliary array for merging)

Merge Sort – Step-by-Step Example



Binary Search

DaC Example 2

Binary Search

Idea: Search a **sorted** array by repeatedly halving the search space.

```
BINARY-SEARCH(A, p, r, key)
  if p > r
    return NOT_FOUND
  mid = floor((p + r) / 2)
  if A[mid] == key
    return mid
  else if A[mid] > key
    return BINARY-SEARCH(A, p, mid-1, key)
  else
    return BINARY-SEARCH(A, mid+1, r, key)
```

DaC structure:

- **Divide:** Compare key with middle element
- **Conquer:** Recurse on one half only
- **Combine:** Trivial (just return the result)

Binary Search – Analysis

Recurrence:

$$T(n) = T(n/2) + O(1)$$

- Only **one** subproblem of size $n/2$
- $O(1)$ work per level (one comparison)

Applying Master Theorem: $a = 1, b = 2, f(n) = O(1)$

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1$$

$$f(n) = O(1) = \Theta(n^{\log_b a}) \implies \text{Case 2}$$

$$T(n) = \Theta(\log n)$$

Binary Search — Step-by-Step Example

Search for key = **23** in sorted array:

```
Index:  0   1   2   3   4   5   6   7   8   9
Value: [3,  8, 11, 15, 20, 23, 29, 31, 48, 65]

Step 1: p=0, r=9, mid=4 → A[4]=20 < 23 → search right
        [                    23, 29, 31, 48, 65]
Step 2: p=5, r=9, mid=7 → A[7]=31 > 23 → search left
        [23, 29]
Step 3: p=5, r=6, mid=5 → A[5]=23 == 23 → FOUND at index 5
```

Only **3 comparisons** for 10 elements ($\lceil \log_2 10 \rceil = 4$ max)

Quick Sort

DaC Example 3

Quick Sort – Overview

Divide: Partition array around a **pivot** element

Conquer: Recursively sort the two subarrays

Combine: Trivial (array is already sorted in-place after partitioning)

```
QUICKSORT(A, p, r)
  if p < r
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q-1)    // sort left of pivot
    QUICKSORT(A, q+1, r)    // sort right of pivot
```

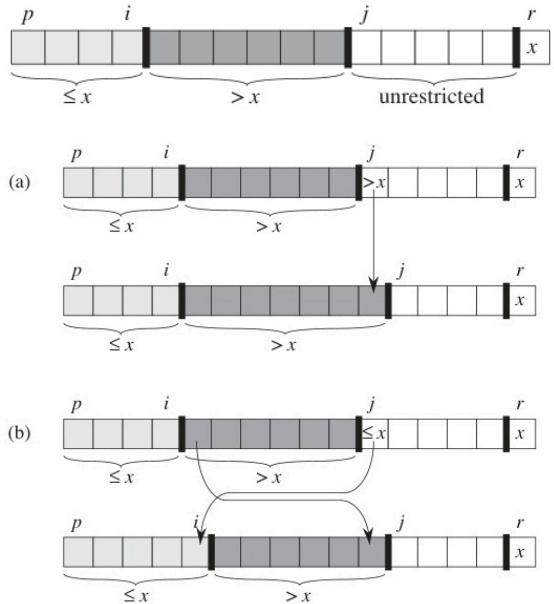
Partition Procedure

Choose the **last element** as pivot. Rearrange so that:

- Elements $\leq$ pivot go to the left
- Elements $>$ pivot go to the right

```
PARTITION(A, p, r)
  pivot = A[r]
  i = p - 1
  for j = p to r - 1
    if A[j] ≤ pivot
      i = i + 1
      swap A[i] and A[j]
  swap A[i+1] and A[r]
  return i + 1
```

Returns the final index of the pivot.



CLRS Figures 7.2 & 7.3

Partition — Step-by-Step Example

Partition `A = [31, 8, 48, 73, 11, 3, 20, 29, 65, 15]` with pivot = **15**

```
pivot = A[9] = 15,  i = -1

j=0: A[0]=31 > 15         → skip           [31, 8, 48, 73, 11, 3, 20, 29, 65, 15]
j=1: A[1]=8  ≤ 15 → i=0  → swap(A[0],A[1]) [ 8, 31, 48, 73, 11, 3, 20, 29, 65, 15]
j=2: A[2]=48 > 15         → skip           [ 8, 31, 48, 73, 11, 3, 20, 29, 65, 15]
j=3: A[3]=73 > 15         → skip           [ 8, 31, 48, 73, 11, 3, 20, 29, 65, 15]
j=4: A[4]=11 ≤ 15 → i=1  → swap(A[1],A[4]) [ 8, 11, 48, 73, 31, 3, 20, 29, 65, 15]
j=5: A[5]=3  ≤ 15 → i=2  → swap(A[2],A[5]) [ 8, 11,  3, 73, 31, 48, 20, 29, 65, 15]
j=6: A[6]=20 > 15         → skip           [ 8, 11,  3, 73, 31, 48, 20, 29, 65, 15]
j=7: A[7]=29 > 15         → skip           [ 8, 11,  3, 73, 31, 48, 20, 29, 65, 15]
j=8: A[8]=65 > 15         → skip           [ 8, 11,  3, 73, 31, 48, 20, 29, 65, 15]

Final: swap A[i+1]=A[3] with A[r]=A[9]:
       [ 8, 11,  3, 15, 31, 48, 20, 29, 65, 73]
                   ^pivot at index 3
```

Result: `[8, 11, 3 | 15 | 31, 48, 20, 29, 65, 73]`

Quick Sort – Analysis

Best/Average case:

- Pivot splits array roughly in half each time

$$T(n) = 2T(n/2) + \Theta(n) \implies T(n) = \Theta(n \log n)$$

Worst case:

- Pivot is always the smallest or largest element
- Partition splits into sizes 0 and $n - 1$

$$T(n) = T(n-1) + \Theta(n) \implies T(n) = \Theta(n^2)$$

Average case (formal):

$$T(n) = \frac{1}{n} \sum_{i=0}^{n-1} [T(i) + T(n-1-i)] + \Theta(n) = \Theta(n \log n)$$

Space: $O(\log n)$ average (recursion stack), $O(n)$ worst case

Selection Problem

Finding the k-th Smallest Element

Selection Problem – Definition

Problem: Given an unsorted array $A[p \ldots r]$, find the i-th smallest element.

Two algorithms:

1. Average-case $\Theta(n)$ – Randomized Select
2. Worst-case $\Theta(n)$ – Median of Medians (Linear Select)

DaC structure:

- **Divide:** Partition array with a pivot, check pivot's rank
- **Conquer:** Recurse on **one** subarray only
- **Combine:** Trivial

$$T(n) \leq \max\{T(k-1),\ T(n-k)\} + \Theta(n)$$

Randomized Select

```
SELECT(A, p, r, i)
  // Find the i-th smallest element in A[p..r]
  if p == r
    return A[p]            // only one element
  q = PARTITION(A, p, r)   // pivot is now at index q
  k = q - p + 1            // pivot is the k-th smallest in A[p..r]
  if i < k
    return SELECT(A, p, q-1, i)       // search left
  else if i == k
    return A[q]                        // pivot is the answer
  else
    return SELECT(A, q+1, r, i-k)     // search right
```

- **Average time:** $\Theta(n)$
- **Worst-case time:** $\Theta(n^2)$

Selection – Example 1: Find the 2nd Smallest

```
Input: [31, 8, 48, 73, 11, 3, 20, 29, 65, 15]   Find: i=2

Step 1: PARTITION with pivot=15
  Result: [8, 11, 3 | 15 | 31, 48, 20, 29, 65, 73]
  pivot at position k=4 (4th smallest)

  i=2 < k=4 → search LEFT group [8, 11, 3]

Step 2: PARTITION [8, 11, 3] with pivot=3
  Result: [3 | 8, 11]
  pivot at position k=1

  i=2 > k=1 → search RIGHT group [8, 11], looking for (2-1)=1st smallest

Step 3: PARTITION [8, 11] with pivot=11
  Result: [8 | 11]
  pivot at position k=2

  i=1 < k=2 → search LEFT group [8]

Step 4: Only one element → return 8
```

Selection – Example 2: Find the 7th Smallest

```
Input: [31, 8, 48, 73, 11, 3, 20, 29, 65, 15]   Find: i=7

Step 1: PARTITION with pivot=15
  Result: [8, 11, 3 | 15 | 31, 48, 20, 29, 65, 73]
  Left group has 3 elements, pivot is k=4

  i=7 > k=4 → search RIGHT group [31, 48, 20, 29, 65, 73]
              looking for (7-4)=3rd smallest in right group

Step 2: Continue recursively in [31, 48, 20, 29, 65, 73]
  to find the 3rd smallest element ...
```

Selection – Average-Case Analysis

Assume the element we seek is always in the **larger** partition (worst scenario for average):

$$T(n) \leq T(3n/4) + \Theta(n)$$

Expanding:

$$T(n) \leq cn + c \cdot \frac{3n}{4} + c \cdot \left(\frac{3}{4}\right)^2 n + \cdots$$

$$= cn \sum_{k=0}^{\infty} \left(\frac{3}{4}\right)^k = cn \cdot \frac{1}{1 - 3/4} = 4cn$$

$$\therefore T(n) = O(n)$$

Since $T(n) = \Omega(n)$ trivially (must examine each element), we get:

$$T(n) = \Theta(n)$$

Selection – Worst-Case Analysis

Worst case: Partition always produces splits of 0 and $n-1$

$$T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$$

Can we guarantee linear time even in the worst case?

Yes! Use the **Median-of-Medians** algorithm.

Linear-Time Selection (Median of Medians)

```
LINEAR-SELECT(A, p, r, i)
  // Find i-th smallest in A[p..r]
  1. if |A| ≤ 5: sort and return the i-th element
  2. Divide elements into groups of 5 → ceil(n/5) groups
  3. Find the median of each group → m_1, m_2, ..., m_{ceil(n/5)}
  4. M = LINEAR-SELECT(medians, 1, ceil(n/5), ceil(n/10))
     // recursively find the median of the medians
  5. Partition A around M
  6. Recurse into the appropriate side
```

Key insight: M is guaranteed to be a **balanced pivot** – at least $3n/10$ elements are smaller and at least $3n/10$ are larger.

Median of Medians — Step-by-Step

Step 2: Divide 37 elements into groups of 5:

```
Group 1: [5, 1, 2, 9, 24]    Group 5: [34, 6, 20, 32, 4]
Group 2: [17, 33, 18, 16, 26] Group 6: [35, 15, 25, 11, 8]
Group 3: [30, 13, 10, 21, 29] Group 7: [28, 23, 27, 22, 19]
Group 4: [3, 36, 7, 37, 12]   Group 8: [31, 14]  (fewer than 5)
```

Step 3: Sort each group, take the median (3rd element):

```
Group 1: [1,2,5,9,24]   → median = 5
Group 2: [16,17,18,26,33] → median = 18
Group 3: [10,13,21,29,30] → median = 21
Group 4: [3,7,12,36,37]   → median = 12
Group 5: [4,6,20,32,34]   → median = 20
Group 6: [8,11,15,25,35]  → median = 15
Group 7: [19,22,23,27,28] → median = 23
Group 8: [14,31]          → median = 14
```

Median of Medians – Continued

Step 4: Find the median of $\{5, 18, 21, 12, 20, 15, 23, 14\}$

Recursively call LINEAR-SELECT on these 8 medians to find the 4th smallest:

$$M = 18$$

Step 5: Partition the entire array around $M = 18$:

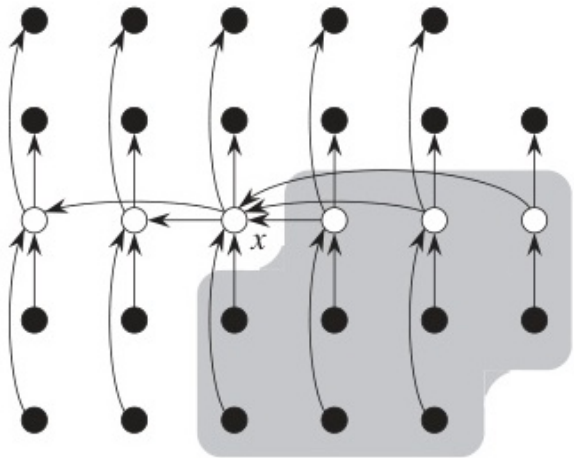
- Left group (≤ 18): elements smaller than or equal to 18
- Right group (> 18): elements larger than 18

Step 6: Recurse into whichever side contains the desired rank.

Why Does Median of Medians Guarantee Balance?

- At least half of the $\lceil n/5 \rceil$ medians are $\leq M$
- For each such median, at least 3 elements in its group are $\leq M$
- So at least $3 \times \lceil n/10 \rceil \approx 3n/10$ elements are $\leq M$
- Similarly, at least $3n/10$ elements are $\geq M$
- **Worst case:** recurse on at most $7n/10$ elements

Each column = group of 5. White circles = medians. Shaded region = elements guaranteed $\geq x$.



CLRS Figure 9.1

Linear Select – Time Complexity

Recurrence:

$$T(n) = T(n/5) + T(7n/10) + \Theta(n)$$

- $T(n/5)$: finding the median of the medians (Step 4)
- $T(7n/10)$: recursing on the larger partition (Step 6)
- $\Theta(n)$: dividing into groups, finding group medians, partitioning

Since $1/5 + 7/10 = 9/10 < 1$, the total work decreases geometrically:

$$T(n) \leq cn + \frac{9}{10}cn + \left(\frac{9}{10}\right)^2 cn + \cdots = cn \cdot \frac{1}{1 - 9/10} = 10cn$$

$$\therefore T(n) = O(n)$$

Since $T(n) = \Omega(n)$ trivially: $T(n) = \Theta(n)$

Closest Pair Problem

DaC Example 4

Closest Pair – Problem Definition

Problem: Given n points on a 2D plane, find the pair of points with the **smallest distance**.

Brute-force approach:

- Compute distance for every pair: $\binom{n}{2} = n(n-1)/2$ pairs
- Each distance computation: $O(1)$
- **Total:** $O(n^2)$

Can we do better with divide-and-conquer?

Closest Pair – DaC Approach

Preprocessing: Sort all points by x-coordinate – $O(n \log n)$

Divide: Split the point set S into left half S_L and right half S_R

Conquer: Recursively find closest pairs CP_L in S_L and CP_R in S_R

Combine: Let $d = \min(\text{dist}(CP_L), \text{dist}(CP_R))$

- Check the **middle strip** (width $2d$) for pairs closer than d
- This pair, if it exists, is CP_C

Return: The closest among CP_L, CP_R, CP_C

Closest Pair – The Middle Strip

The critical insight is in the **combine** step:

```
←— d —→←— d —→
┌──────┬──────┐
│ S_L  │ S_R  │
│ · ·  │ ·    │
│   ·  │   ·  │
└──────┴──────┘
Middle strip (width 2d)
```

- Only points within distance d of the dividing line need checking
- Sort strip points by y-coordinate
- For each point, compare with only the next few points (at most **6** neighbors)
- This makes the strip check $O(n)$ per point but with constant-bounded comparisons

Closest Pair – Pseudocode

```
CLOSEST-PAIR(S)
  Input: S — points sorted by x-coordinate
  Output: closest pair distance

  1. if |S| ≤ 3: compute all pairwise distances, return min
  2. Split S into S_L and S_R at the median x-coordinate
  3. CP_L = CLOSEST-PAIR(S_L)
  4. CP_R = CLOSEST-PAIR(S_R)
  5. d = min(dist(CP_L), dist(CP_R))
     Find closest pair CP_C among points in the middle strip
     (within distance d of the dividing line)
  6. return min(CP_L, CP_C, CP_R) by distance
```

Closest Pair – Execution Example

```
Points sorted by x: (2,·) (5,·) (10,·) (15,·) (20,·) | (25,·) (26,·) (28,·) (30,·) (37,·)
                    ←──────── S_L ────────→           ←──────── S_R ────────→

Step 1: Recurse on S_L → CP_L with dist = 10
Step 2: Recurse on S_R → CP_R with dist = 15
Step 3: d = min(10, 15) = 10

Step 4: Middle strip = points with x in [20-10, 25+10] = [10, 35]
        Strip points: (10,·) (15,·) (20,·) (25,·) (26,·) (28,·) (30,·)

Step 5: Sort strip by y-coordinate, check nearby pairs
        → Find CP_C with dist = 5 (for example)

Step 6: return min(10, 5, 15) = 5 → CP_C is the closest pair
```

Closest Pair – Time Complexity

Recurrence analysis:

Line	Cost
Line 1 (base case)	$O(1)$
Line 2 (split)	$O(1)$ (already sorted)
Lines 3–4 (recurse)	$2T(n/2)$
Line 5 (strip sort + scan)	$O(n \log n)$
Line 6 (return)	$O(1)$

$$T(n) = 2T(n/2) + O(n \log n)$$

By the Master Theorem or direct analysis:

When DaC is Inappropriate

When DaC Fails – Fibonacci Example

DaC is inappropriate when: the total input size of subproblems **grows** after division.

Fibonacci: $F(n) = F(n-1) + F(n-2)$

```
                F(6)
               /    \
           F(5)      F(4)
          /    \     /    \
      F(4)   F(3)  F(3)  F(2)
     /   \   / \   / \
  F(3) F(2)...   ...
```

- Input size: n
- Subproblem input sizes: $(n-1) + (n-2) = 2n - 3 > n$
- $F(2)$ computed **5 times**, $F(3)$ computed **3 times**
- Exponential redundancy!

Fibonacci – Bottom-Up Solution

Instead of DaC, use **iterative bottom-up** computation:

```
FIB-NUMBER(n)
  F[0] = 0
  F[1] = 1
  for i = 2 to n
    F[i] = F[i-1] + F[i-2]
  return F[n]
```

Time: $\Theta(n)$ – each value computed exactly once

Lesson: When subproblems overlap significantly, **Dynamic Programming** (bottom-up or memoization) is more appropriate than naive divide-and-conquer.

Cautions When Applying DaC

Two key considerations:

1. **Input size growth:** If the total size of subproblems exceeds the original input size, DaC leads to exponential blowup
   - Example: Fibonacci
2. **Combine cost matters:** Simply dividing input does not guarantee efficiency
   - The cost of combining subsolutions must be manageable
   - Many geometry problems work well with DaC because their combine steps naturally fit the problem structure

Summary

Summary

Algorithm	Recurrence	Time Complexity
Merge Sort	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
Binary Search	$T(n) = T(n/2) + O(1)$	$\Theta(\log n)$
Quick Sort (avg)	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
Quick Sort (worst)	$T(n) = T(n-1) + \Theta(n)$	$\Theta(n^2)$
Selection (avg)	$T(n) \leq T(3n/4) + \Theta(n)$	$\Theta(n)$
Selection (worst, MoM)	$T(n) = T(n/5) + T(7n/10) + \Theta(n)$	$\Theta(n)$
Closest Pair	$T(n) = 2T(n/2) + O(n \log n)$	$O(n \log^2 n)$

Q & A

codingchild@korea.ac.kr